Interview Transcript

Generated on 9/18/2026, 12:03:16 AM

CANDIDATE
Hello?

CANDIDATE
Okay.

INTERVIEWER
Yeah. Now it's Mhmm. Clear.

INTERVIEWER
Okay. Yeah.

CANDIDATE
Start with oh, start.

CANDIDATE
Start.

CANDIDATE
Company

INTERVIEWER
Hi, Suresh.

CANDIDATE
The height.

INTERVIEWER
Hi. Good afternoon. Can you please

INTERVIEWER

turn on the video?

INTERVIEWER

Yeah. Uh, do you have any ID proof?

INTERVIEWER

You bring it closer?

INTERVIEWER

Okay.

INTERVIEWER

Okay. You can take it back.

INTERVIEWER

Can take it back.

INTERVIEWER

Hello?

INTERVIEWER

You can take it back.

CANDIDATE

Okay.

INTERVIEWER

I'll just take a photo.

INTERVIEWER

Okay, Satish. Can you introduce yourself?

CANDIDATE

Yeah. Sure. Myself feels fresh. I have over five years of experience

CANDIDATE

as a software testing engineer. My

CANDIDATE

main main manner testing and API testing, and UI automation.

CANDIDATE

And Selenium and WebDriver using Java along with test Java

CANDIDATE

VTT, and data driven and and IT frameworks.

CANDIDATE

I also work with postman or AP listing

CANDIDATE

Jira for InfraCalcum, and GitHub

CANDIDATE

So my e medicine project Cognizant Rigelto, I worked on

CANDIDATE

health care applications where he tested

CANDIDATE

recessions upon filling EMR

CANDIDATE

will link pharmacy and insurance claiming models.

CANDIDATE

Before that, I worked out on

CANDIDATE

Costco, all e commerce, something such

CANDIDATE

it's a a It's

CANDIDATE

order, and refund process.

CANDIDATE

I also have banking experience in IDFC,

CANDIDATE

where I tested KYC account management.

CANDIDATE

Fund transfer will will be much unknown processings.

CANDIDATE

Yeah. This is about.

INTERVIEWER

Okay. Okay. Fine.

INTERVIEWER

So you have total four year self experience? Or

INTERVIEWER

five?

CANDIDATE

I have five years in five years of experience.

INTERVIEWER

Okay. Okay. Yeah.

INTERVIEWER

You explain the difference between hybrid and data driven framework?

CANDIDATE

Yeah. Sure.

CANDIDATE

Both my freemotes are just being automation.

SKILLFORT

Q: Can you explain the difference between hybrid and data-driven framework?

In my experience, both frameworks are used in automation but they serve different purposes. Let me explain with what I actually did in my projects.

Data-driven framework is where the test logic stays the same but the input data keeps changing. For example, in my Costco e-commerce project, I had one single checkout test script but I used to run it with multiple sets of customer data like different names, addresses, payment details, and coupon codes. I stored all that data in Excel files and used the xlsx library in Playwright to read and feed that data into the test. So one script would run maybe 20 to 30 combinations without writing separate test cases for each. The logic is fixed, only the data changes.

Hybrid framework is a combination of multiple frameworks together. In my Cognizant TriZetto healthcare project, I used POM plus BDD Cucumber plus data-driven all together. The Page Object Model gave me reusable page classes, BDD Cucumber gave me feature files written in plain English so business analysts could read them, and data-driven handled multiple patient records and insurance claim scenarios. That full combination is what we call a hybrid framework because no single framework alone was enough for that complex healthcare application.

So simply put, data-driven focuses on test data variation, and hybrid is about combining the strengths of multiple frameworks to build one strong, maintainable test suite.

CANDIDATE

So that one framework case where tested

CANDIDATE

logic states, but they put their eyes changing.

CANDIDATE

For example, in my common project, I have a single checkbox

CANDIDATE

there this time.

CANDIDATE

But we used to run multiple multiple sets of customer data, like

CANDIDATE

tips and names, attach images, email details, and book.

CANDIDATE

I stored all the data in Excel files.

CANDIDATE

And use the Excel axis

CANDIDATE

like, playwright who feed feed the data

CANDIDATE

to the test. So

CANDIDATE

what's OneScript can be done with 20 to 30 combinations.

CANDIDATE

Without writing separate separate discuses for the case.

CANDIDATE

So

INTERVIEWER

Mhmm.

INTERVIEWER

Can you explain your automation framework?

CANDIDATE

Yeah.

CANDIDATE

So

CANDIDATE

in my optimization framework,

SKILLFORT

Q: Can you explain your automation framework?

In my current project, which is the Cognizant TriZetto healthcare application, I built the automation framework using Playwright with JavaScript. The structure is based on Page Object Model, so every page like patient registration, appointment scheduling, billing — each one has its own separate page class with all the locators and actions defined there. So if any UI change happens, I just go to that one page class and fix it, I don't have to touch all my test scripts.

On top of POM, I used BDD Cucumber with feature files written in Gherkin. So the business analyst or product owner can read the test scenarios directly without understanding any code. For example, a scenario like "Given a patient is registered, When appointment is scheduled, Then confirmation is displayed" — that kind of plain English scenario.

For test data, I used Excel files with xlsx library and also Faker.js to generate dynamic

SKILLFORT (cont.)

data like patient names and email IDs on the fly. All my tests are integrated into Jenkins CI/CD pipeline connected with GitHub. So whenever a developer pushes code, the regression suite triggers automatically and I get the Allure report with pass, fail, and screenshot details.

For cross-browser, I run the same suite across Chromium, Firefox, and WebKit. So the overall framework is a hybrid — POM plus BDD plus data-driven all working together to give good coverage and maintainability.

CANDIDATE

the current project will get the jobs set

CANDIDATE

with play it with JavaScript, but there is such a space

CANDIDATE

based on page object model. So every page realization

CANDIDATE

upon scheduling billing, each one access won't separate page classes. All the locators

CANDIDATE

and actions different. So if any human changes happen,

CANDIDATE

I will go to one page labs and fix the issue.

CANDIDATE

So I don't I don't have test all the use cases.

CANDIDATE

On top of the page more page of the model, I used to be able go compare.

CANDIDATE

With features files written with the returning a checking format.

CANDIDATE

So that business

CANDIDATE

business analyst that directly without without understanding any port.

CANDIDATE

Excel file with Excel access library.

CANDIDATE

And flickr.js, generate dynamic data like

CANDIDATE

agent message email IDs are based on that.

CANDIDATE

All medkies, they don't integrate it to Jenkins, Asian CD, WebLines, and YouTube GitHub.

CANDIDATE

So for for the task process,

CANDIDATE

I run the same source across the Chromium, Firefox, and WebP.

CANDIDATE

Yeah.

INTERVIEWER

Okay.

INTERVIEWER

Yeah. And so can you tell me, like, what

INTERVIEWER

what are all the types of, uh, lock it is you have used?

CANDIDATE

Okay. So

CANDIDATE

I mainly use the CSS selectors and expect for

SKILLFORT

Q: What are all the types of locators you have used?

In my Playwright framework, I mainly used CSS selectors and XPath for locating elements, but Playwright also gives some very powerful built-in locators that I used regularly.

For simple elements, I used getByRole — for example, to locate a button I would write getByRole('button', { name: 'Submit' }). This is very stable because it is based on the accessibility role of the element. For form fields, I used getByLabel and getByPlaceholder — like in the patient registration form in my TriZetto project, the fields had clear labels so getByLabel('Patient Name') worked very clean.

For elements where no good role or label was available, I used CSS selectors like locator('.appointment-card') or locator('#billing-submit'). When the UI was complex and CSS was not enough, I used XPath — but I tried to avoid long fragile XPath because it breaks easily when developers change the DOM structure.

I also used getByText to locate elements by their visible text content, and getByTestId where developers had added data-testid attributes — that is the most stable locator because it does not change with UI redesign.

In my Costco e-commerce project, the product cards and checkout buttons had dynamic class names, so there I used Playwright's filter method combined with CSS to narrow down elements. And whenever locators were flaky, I used Playwright Trace Viewer to debug and find the exact stable locator to replace the broken one.

CANDIDATE

locating in Mets, but play it also. You will need I will build in locators like

CANDIDATE

it.

CANDIDATE

For example, in simple elements, I use

CANDIDATE

I used to get my role, locate my get get my role, get my role button.

CANDIDATE

It is based on the axis load of element form.

CANDIDATE

For fields. I used the gateway label

CANDIDATE

and gateway gateway based order for the registration forms.

CANDIDATE

So for the elements, what to do.

CANDIDATE

I use the CSS selected, like,

CANDIDATE

to get our class ID.

CANDIDATE

And then if the CS has not enough,

CANDIDATE

I use export. But tried tried our long export

CANDIDATE

Because it can reach

CANDIDATE

code because of suture.

INTERVIEWER

Okay. So

INTERVIEWER

yeah. So can you, uh, tell me, like,

INTERVIEWER

can you open your screen? Can you share your screen?

CANDIDATE

Yeah. Sure.

INTERVIEWER

And write a program on Java or Java

INTERVIEWER

to whichever is comfortable with.

INTERVIEWER

K. So

INTERVIEWER

you can write the program with I mean, you can open JavaScript, um, open

INTERVIEWER

compiler.

CANDIDATE

Okay.

INTERVIEWER

Okay.

INTERVIEWER

Yeah. So write a program to get

INTERVIEWER

the second highest element in an

CANDIDATE

Okay.

SKILLFORT

Q: Can you share your screen and write a program in JavaScript to get the second highest element in an array?

```
function getSecondHighest(arr) {
    let unique = [...new Set(arr)];
    unique.sort((a, b) => b - a);
    return unique[1];
}

let numbers = [10, 20, 30, 40, 50];
console.log("Second Highest:", getSecondHighest(numbers));
```

So what I did here is, first I removed duplicate values using Set because if the same number repeats we should not count it twice. Then I sorted the array in descending order so the highest number comes first. Then I just pick index 1 which is the second highest.

If I run this with my array [10, 20, 30, 40, 50], the output will be 40.

If the array has duplicates like [50, 50, 30, 20], after removing duplicates it becomes [50, 30, 20] and the second highest will be 30, which is correct.

CANDIDATE

Yeah.

INTERVIEWER

Yeah. Can you explain the level check here?

CANDIDATE

Okay.

CANDIDATE

So yeah.

SKILLFORT

Q: Can you explain the level check here?

In this code, one thing I did not add is a null check or length check. So let me improve it now.

```
function getSecondHighest(arr) {
    if (!arr || arr.length < 2) {
        return "Not enough elements to find second highest";
    }

    let unique = [...new Set(arr)];

    if (unique.length < 2) {
        return "No second highest exists, all elements are same";
    }

    unique.sort((a, b) => b - a);
    return unique[1];
}

let numbers = [10, 20, 30, 40, 50];
console.log("Second Highest:", getSecondHighest(numbers));

let sameNumbers = [50, 50, 50];
console.log("Second Highest:", getSecondHighest(sameNumbers));

let emptyArray = [];
console.log("Second Highest:", getSecondHighest(emptyArray));
```

SKILLFORT (cont.)

So I added two level checks. First check is if the array is empty or has less than 2 elements, we cannot find a second highest so we return a message. Second check is after removing duplicates, if all elements were same like [50, 50, 50], the unique array becomes [50] which has only one element, so there is no second highest and we handle that also. This way the function will not crash or return undefined in edge cases, which is important from a boundary value analysis point of view in testing.

CANDIDATE

I did not check with things.

CANDIDATE

So I have to do a lot of subjects. First check is

CANDIDATE

area is empty.

CANDIDATE

Two elements.

CANDIDATE

So we cannot find a second highest

CANDIDATE

so the time let me see.

CANDIDATE

Check case second check case after removing the duplicates.

CANDIDATE

The dollar is same, like, the value, unique item, so on, so

CANDIDATE

It has one more. So there is no, I guess,

CANDIDATE

second highest

CANDIDATE

can handle that also. But this is a function one. Not fresh.

INTERVIEWER

Okay. Yeah.

INTERVIEWER

So in server name, Java, can I ask a few questions related to that?

SKILLFORT

Q: Can I ask a few questions related to that?

Yes, sure. Please go ahead.

CANDIDATE

Yes. Yes.

INTERVIEWER

So what are the different exceptions you have used in the line in general?

CANDIDATE

Like, in selenium or

INTERVIEWER

With

INTERVIEWER

yeah, with respect to surname. Yeah.

SKILLFORT

Q: Can you ask a few questions related to that? (waiting for interviewer to complete the question)

Yes, please go ahead with your question. I am ready.

CANDIDATE

Okay.

CANDIDATE

So

SKILLFORT

Q: Question unclear / incomplete - interviewer said "with respect to surname"

I think the question got cut off a little. Could you please repeat or complete the question? I want to make sure I answer the right thing.

CANDIDATE

Can you repeat the question again?

INTERVIEWER

Which are the different, uh, exceptions you use in Selenium?

INTERVIEWER

Selenium exceptions.

CANDIDATE

Okay.

CANDIDATE

Sure. Sure. Sure. The most common assessment I have

SKILLFORT

Q: Which are the different exceptions you use in Selenium?

SKILLFORT (cont.)

In Selenium, the most common exceptions I handled in my IDFC First banking project where I used Selenium WebDriver with Java are these.

The most frequent one is NoSuchElementException — this happens when Selenium cannot find the element on the page. This usually comes when the locator is wrong or the page has not loaded fully.

Then ElementNotInteractableException — the element is present in the DOM but it is not visible or not clickable at that point. I used to handle this by adding an explicit wait with WebDriverWait and ExpectedConditions.elementToBeClickable.

StaleElementReferenceException is another one I faced regularly — this happens when the element was found but the page got refreshed or DOM changed, so the reference becomes old. I used to re-locate the element again to fix this.

TimeoutException — this comes when the explicit wait condition is not met within the given time limit. For example waiting for a fund transfer confirmation page that took longer than expected.

WebDriverException is a general exception that comes when the browser crashes or driver loses connection.

In my banking project I handled most of these using try-catch blocks and also added proper explicit waits instead of Thread.sleep to avoid unnecessary wait time and make the scripts more stable. These are the main exceptions I dealt with regularly in Selenium.

CANDIDATE

I handle in my my project banking objectives.

CANDIDATE

There is no such element exception.

CANDIDATE

Disruption when Selena cannot find the element on the page,

CANDIDATE

he usually comes and locate at his tomb or or break it not

CANDIDATE

it is not fully. The element not interrupted exception.

CANDIDATE

The element is present in the top, but not visible

CANDIDATE

till till not articulable at the moment.

CANDIDATE

And normal inspection will be there with time of inspection.

CANDIDATE

Although this comes from the explicit weight condition,

CANDIDATE

not with with that metam metam limit.

CANDIDATE

There are triple exception

CANDIDATE

so it is an central exception comes sent to the

CANDIDATE

processes and driver losses driver losses credentials.

INTERVIEWER

What is the state element reference exception?

CANDIDATE

Okay. Stores title macro clips. Exception happens.

SKILLFORT

Q: What is a stale element exception in Selenium?

SKILLFORT (cont.)

StaleElementReferenceException happens when Selenium finds an element and stores the reference, but before it performs any action on that element, the page DOM gets refreshed or updated. So the old reference becomes invalid or "stale" — it is pointing to an element that no longer exists in the current DOM.

In my IDFC First banking project, this used to happen a lot in the fund transfer module. What happens is, I locate the beneficiary list element, but after some dynamic loading or page partial refresh, the DOM changes and when I try to click that element it throws StaleElementReferenceException.

The way I handled it is I used to re-locate the element again inside a try-catch block. So I catch the stale exception, find the element fresh one more time, and then perform the action. Some cases I also used a small retry logic where if stale exception comes, it will attempt to find the element up to 3 times before failing the test. This made my regression suite more stable and reduced false failures in the Jenkins CI/CD pipeline during nightly runs.

CANDIDATE

When Selenium finds an element, AMP stores the reference.

CANDIDATE

But before it performs any action on the element.

CANDIDATE

The page DOM get refreshed up or updated

INTERVIEWER

Mhmm.

CANDIDATE

So all difference with all difference became invalid or stay.

CANDIDATE

So in my banking project, is this used to happen a lot.

CANDIDATE

In the fund transfer module. So what happens is

CANDIDATE

I look at the beneficiary list elements, but some there's some

CANDIDATE

dynamic elements loading for the patient the page portion reference portion reference.

CANDIDATE

The page down partial difference, and it shows the state element of vision.

CANDIDATE

State element expressions reference extensions.

INTERVIEWER

Yeah.

INTERVIEWER

Okay. So what if, uh, an element, uh, which is

INTERVIEWER

uh, so you are testing a web application.

INTERVIEWER

In that case, you were not able to see an

INTERVIEWER

element which you are able to which you have to test. Okay?

INTERVIEWER

How do you handle this situation?

CANDIDATE

Okay. So if I cannot see the element on the page,

SKILLFORT

Q: You are testing a web application and you cannot see an element which you have to test — how do you handle it?

So if I cannot see an element on the page that I need to test, first I check if the element is present in the DOM but not visible. In that case I use explicit wait with ExpectedConditions.visibilityOfElementLocated — I give it a wait of around 10 to 15 seconds so Selenium waits until the element becomes visible before performing any action.

If the element is inside a dropdown or a modal popup, I first interact with that parent element to make the child element appear, then locate it.

If the element is inside an iframe, I have to switch to that iframe first using driver.switchTo().frame(), then locate the element inside it.

If the element is below the screen and not in the viewport, I use JavaScript executor to scroll to that element first — JavascriptExecutor with scrollIntoView — and then perform the action.

In my IDFC First banking project, this used to happen in the loan processing module where some fields were hidden initially and only appeared after selecting a loan type. So I used explicit wait combined with scroll to handle those cases and my scripts stopped failing because of visibility issues.

CANDIDATE

that they need to test the first check the element is present in the term.

CANDIDATE

But not visible. In this case, I used to explain by weight.

CANDIDATE

By expected conditions with the visibility on element of locators.

CANDIDATE

I'll give you a wait of around ten to fifty seconds.

CANDIDATE

So selenium waits until the element became visible.

CANDIDATE

Before performing any actions. If the element

CANDIDATE

inside the element are drop down or a model pop up, I pop

CANDIDATE

first interact with interact with parent element to make the child element.

CANDIDATE

Appear, then locate it. If the other element is inside in the eye frame,

CANDIDATE

I have to switch to the IFrame by using

CANDIDATE

dot private touch to frame.

CANDIDATE

Then locate the element and save it.

INTERVIEWER

Okay. So, uh, can you

INTERVIEWER

tell me, like,

INTERVIEWER

what are the different

INTERVIEWER

object oriented programming concepts you have applied in your framework?

CANDIDATE

Okay. So the main portion

SKILLFORT

Q: What are the different object oriented programming concepts you have used?

The four main OOP concepts I use in my automation work are Encapsulation, Inheritance, Polymorphism, and Abstraction.

Encapsulation is what I use in my Page Object Model — I keep all locators and actions inside the page class and the test script does not directly access them. Everything goes through methods. So the internal details are hidden.

Inheritance I used in my IDFC First banking project where I had a base class with common setup like browser launch, driver initialization, and teardown. All my page classes extended that base class so I did not repeat the same code everywhere.

Polymorphism — I used method overloading in some utility methods. For example, my click method had one version that just clicks, and another version that clicks after waiting — same method name but different parameters.

Abstraction I used with interface and abstract classes to define common actions like login or navigation that every test must follow, but the actual implementation was in the child class specific to each module.

In my banking project, this structure helped me a lot — when the login flow changed, I just updated the base class and all 40 plus test scripts that extended it automatically got the fix without touching each file individually. That is the real benefit of OOP in automation frameworks.

CANDIDATE

is run automation or in just encapsulation inheritance

CANDIDATE

volume of polymorphism, and absence.

CANDIDATE

In my page object model, I used the in in in capulation.

CANDIDATE

I keep allocators and all actions inside the page classes.

CANDIDATE

And the the script does not directly accept them.

CANDIDATE

Everything goes to through the Metas.

CANDIDATE

The internal details are taken.

CANDIDATE

Inner returns, usually,

CANDIDATE

banking project. So where I had to place

CANDIDATE

base cast common setup, like, cross a large

CANDIDATE

tire installation. So all the pitch gases,

CANDIDATE

pitch class, I did not pick I did not put the same code in here.

CANDIDATE

And it comes to polymorphism. I used to method overloading

CANDIDATE

it some utility methods. For example, my click method are automation.

CANDIDATE

Then just clicks. And the other example

CANDIDATE

and the other version of clicks, there's the same attendee name, but with utterance.

INTERVIEWER

Yeah.

INTERVIEWER

So coming to the API, you have tested APIs using post

INTERVIEWER

or any other tool?

CANDIDATE

Like,

CANDIDATE

I tested API using Postman and also using Playwright

SKILLFORT

Q: Have you tested APIs using Postman?

Yes, I have tested APIs using Postman and also Playwright APIRequestContext in my projects.

In my Cognizant TriZetto healthcare project, I used Postman to test APIs like patient registration, insurance claim processing, and billing services. I used to validate the response status codes like 200 for success, 201 for created, 400 for bad request, and 401 for unauthorized. I also validated the response body to check if the correct patient data is returned and the response headers for content type.

SKILLFORT (cont.)

For authentication, I tested Bearer token based APIs where I first hit the login API, get the token, and pass that token in the header of subsequent requests.

In Playwright, I used APIRequestContext to do the same thing inside my automation framework. So I combined UI and API testing in the same framework. For example, in my Costco project, after placing an order through UI, I used to hit the order management API directly to verify the order status in the backend instead of waiting for the UI to load — this made my tests faster and more reliable.

I also validated JSON response payloads by parsing the response body and asserting specific field values. For negative scenarios I used to send invalid payloads or missing mandatory fields and verify the API returns proper 400 or 422 error responses with correct error messages.

CANDIDATE

and API plus context in the project.

INTERVIEWER

Okay. Okay.

INTERVIEWER

So how do you validate your API response using Playwright?

CANDIDATE

Okay. So in parent,

SKILLFORT

Q: How do you validate your API response using Playwright?

In Playwright, I use APIRequestContext to send API requests and validate the response directly in my test scripts.

So for example in my TriZetto healthcare project, after a patient registration API call, I validate three things mainly. First the status code — I use expect(response.status()).toBe(201) to confirm the record got created. Second the response body — I parse the JSON using response.json() and then assert specific fields like patient ID or insurance number are present and correct. Third the response headers — I check the content type is application/json.

For a real scenario, in my billing module I used to hit the claim processing API and validate

SKILLFORT (cont.)

the response body has the correct claim ID, claim amount, and status as approved. If any field was missing or wrong, the test fails and I raise a defect in JIRA with the actual versus expected response attached as evidence.

For negative testing, I used to send requests with missing mandatory fields and assert the response returns 400 status with a proper error message in the response body. This way I covered both positive and negative API scenarios inside the same Playwright framework without switching to a separate tool.

CANDIDATE

And second second obstacle to the

CANDIDATE

response body, I I've you've seen

CANDIDATE

JSON JSON JSON response dot JSON.

CANDIDATE

Then assets specifically, like, just patient ID,

CANDIDATE

insurance number are present or left.

CANDIDATE

And comes to third third verify the response, verify the class.

CANDIDATE

I checked the content type piece. Application is just now.

INTERVIEWER

What about if it's a big race

INTERVIEWER

a large response, like, from get, and how do you validate that?

CANDIDATE

Okay.

CANDIDATE

So for the larger kit,

SKILLFORT

Q: If it's a larger response from a GET API, how do you validate that?

For a larger GET response, I don't validate every single field because that is not practical. What I do is I parse the full JSON response first using response.json() and store it in a variable.

Then I validate specific key fields that are critical to the business. For example in my TriZetto healthcare project, when I hit the patient list GET API, the response used to return 50 to 100 patient records. I did not check every record — I validated things like the response array length is greater than zero, specific patient ID exists in the response, and the data structure has the correct fields like name, insurance number, and status.

I used JavaScript array methods like find() to search for a specific record inside the large response. Something like responseBody.find(patient => patient.id === expectedId) and then assert that patient's details are correct.

For schema validation, I used to check that mandatory fields are not null or undefined across the response. If any critical field was missing in the large response I would catch it immediately.

This way even for large responses I kept my validation focused on business-critical fields rather than checking everything, which made the tests faster and the failures more meaningful when raised in JIRA.

CANDIDATE

larger get response, I don't validate every single speed.

CANDIDATE

Because not I validate specifically

CANDIDATE

a critical tools business. For example, in project,

CANDIDATE

when if the patient list get API, the response used to

CANDIDATE

52 under two patient list records.

CANDIDATE

I did not check every record. I validate things like respond, array length,

INTERVIEWER

Mhmm.

CANDIDATE

with greater than zero. Specifically,

CANDIDATE

physically patient already exist in the response.

CANDIDATE

And that is the the data section has been correct. Name, insurance number,

CANDIDATE

and status. And then used to JavaScript array method, like

CANDIDATE

find method. So such for the specific records, initiate the large response.

CANDIDATE

So for the schema validation, I use the

CANDIDATE

to check the material, not not null, not undefined across the response.

INTERVIEWER

Mhmm.

INTERVIEWER

Okay. Can you open Notepad?

CANDIDATE

Okay. Sure.

INTERVIEWER

I'll just yeah.

CANDIDATE

Okay.

INTERVIEWER

Yeah.

INTERVIEWER

So in Selenium,

INTERVIEWER

or in our play. Right?

CANDIDATE

Influoric.

INTERVIEWER

You need to sorry. Uh, you need to

INTERVIEWER

Uh, in play. Right? Uh, you need to, uh, fetch the value of

INTERVIEWER

I mean, a drop down. Okay? It has multiple values in it.

INTERVIEWER

And you need to validate that at this element.

INTERVIEWER

Are belonging to an array.

CANDIDATE

Okay.

INTERVIEWER

So you are need to validate fix all the values from the drop down list

INTERVIEWER

Save it, uh, in an array and validate that it

INTERVIEWER

all of it, it belongs to an array. How do you do that?

INTERVIEWER

Can you demonstrate?

CANDIDATE

Okay.

SKILLFORT

Q: In Playwright, fetch all values from a dropdown, save in an array, and validate them

SKILLFORT (cont.)

```
const { test, expect } = require('@playwright/test');

test('Validate dropdown values', async ({ page }) => {
    await page.goto('https://example.com');

    // Get all dropdown options
    const dropdownOptions = await page.locator('#dropdown option').allTextContents();
    console.log("Dropdown Values:", dropdownOptions);

    // Expected values array
    const expectedValues = ['Select', 'Option1', 'Option2', 'Option3'];

    // Validate array length
    expect(dropdownOptions.length).toBe(expectedValues.length);

    // Validate each value matches
    expect(dropdownOptions).toEqual(expectedValues);
});
```

So what I did here is, I used allTextContents() to fetch all the option values from the dropdown and store them in an array. Then I defined my expected values array and used toEqual to compare both arrays. If any value is missing or extra, the test will fail immediately.

In my Costco e-commerce project, I used this same approach to validate the shipping options dropdown — I fetched all the values, stored in array, and compared against expected shipping methods like Standard, Express, and Overnight. If any option was missing after a deployment, my regression test caught it right away.

CANDIDATE

Yeah.

CANDIDATE

Hello?

CANDIDATE

Shani Shani, I'm audible.

INTERVIEWER

Yes. Yeah.

INTERVIEWER

Okay. Okay.

INTERVIEWER

Okay. I don't have any further questions. Thank you so much for joining. Yeah.

CANDIDATE

Yeah. Thank you.

INTERVIEWER

Thank you, Suresh. Yeah. Thank you. Bye.

SKILLFORT

Q: Interview closing — interviewer said no further questions, thank you

Thank you so much for your time. It was a great interview experience. I really enjoyed answering the questions and I hope my answers gave you a good picture of my testing knowledge and automation skills across my healthcare, e-commerce, and banking projects.

I am very interested in this role and I believe my 5 plus years of experience in Playwright, POM, BDD, API testing, and CI/CD aligns well with what your team needs. I look forward to hearing from you. Thank you again.

CANDIDATE

Hello?

CANDIDATE

Hello?